AI Quality Gateway Extension — Architecture Spec

Status: Draft v1.0 — 2026-05-31 **Scope:** Additive extension to tas-llm-router enabling the AI Quality Gateway (AIQG) product **Companion docs:** build-vs-reuse plan, 16 AIQG data-model docs

This document specifies how to add the AI Quality Gateway capability to `tas-llm-router` **without forking the repo and without breaking any existing caller**. All new behavior activates only when an inbound request explicitly opts in via headers; the existing internal-routing path is byte-identical to today.

---

1. Goals and Non-Goals

Goals

1. Customer-facing streaming LLM proxy that captures per-request diagnostic data
2. CLEAR (Cost, Latency, Efficacy, Assurance, Reliability) measurement on production traffic
3. Path A authentication — vendor keys flow through; gateway never stores them
4. Per-chunk latency decomposition (DNS, TLS, TTFB, TTFT, inter-token, last-chunk)
5. Workflow classification + tag set + policy resolution per request
6. New `com.tas.aiqg.{request,response}.v1` CloudEvents on `tas.aiqg.*` Kafka topics
7. CLEAR composite scoring at request close (Go, in-process)

Non-Goals

1. **Changing any existing exported types, method signatures, JSON wire shapes, Kafka topics, HTTP endpoints, config keys, or Prometheus metric names.** Per build-vs-reuse §1.2, all changes are additive.
2. Path B (stored vendor keys) — deferred to Phase 2
3. Thin client SDK — Phase 3
4. Payload reduction — Phase 2
5. Route policy editor UI — that lives in `aiqg-ui`, not here

---

2. AIQG-Mode Detection

A single per-request decision determines whether AIQG behavior activates:

```
// internal/aiqg/mode.go
type Mode int

const (
    ModeInternalRouting Mode = iota // existing behavior; vendor key from config
    ModeAIQG                        // Path A; vendor key from inbound header; AIQG capture +
)

func DetectMode(r *http.Request) Mode {
    tasAuth := r.Header.Get("TAS-Auth")
    if tasAuth == "" {
        return ModeInternalRouting
    }
    if !strings.HasPrefix(tasAuth, "tas_qg_live_") &&
```

```
        !strings.HasPrefix(tasAuth, "tas_qg_test_") {
        return ModeInternalRouting
    }
    // Has TAS-Auth with the AIQG token prefix → AIQG mode
    return ModeAIQG
}
```

Mode is computed once at the top of the request handler chain and propagated through context.Context:

```
type modeKey struct{}

func WithMode(ctx context.Context, m Mode) context.Context {
    return context.WithValue(ctx, modeKey{}, m)
}

func ModeFromContext(ctx context.Context) Mode {
    if m, ok := ctx.Value(modeKey{}).(Mode); ok {
        return m
    }
    return ModeInternalRouting
}
```

Every new behavior in this document checks `ModeFromContext(ctx) == ModeAIQG` before executing. Internal-routing requests bypass every AIQG code path entirely — no extra allocations, no extra latency.

---

3. New Package Layout

All new code lands under additive paths. Existing packages are untouched except where called out.

```
tas-llm-router/
├── internal/
│   ├── aiqg/
│   │   ├── mode.go                  # Mode detection + context propagation (§2)
│   │   ├── auth_pathA.go            # tas_qg_* token validation, account lookup
│   │   ├── bearer_context.go        # Per-request bearer override via ctx (§4)
│   │   ├── account_client.go        # Calls aiqg-dashboard-be for account lookup
│   │   └── account_cache.go         # Redis-backed account cache, 60s TTL
│   ├── instrumentation/
│   │   ├── httptrace.go             # net/http/httptrace ClientTrace wrapper (§5)
│   │   ├── timing.go                # TimingCollector keyed by ctx (§5)
│   │   └── chunk_stamp.go           # No-op-when-absent chunk timestamp helper
│   ├── middleware/
│   │   └── aiqg_headers.go        # Parses TAS-* headers; strips before upstream (§6)
│   ├── workflow/
│   │   └── classifier.go         # Calls Gatekeeper scanner with aiqg_workflows.yaml
│   ├── policy/
│   │   ├── resolver.go              # Route-matcher → bundle resolution
│   │   ├── neo4j_client.go          # Neo4j queries for bundles + route rules
│   │   └── cache.go                 # Redis-cached bundle/route resolution
│   ├── sampling/
```

```
│   │   └── stratified.go            # LLM-as-judge sampling decision (§7)
│   ├── judging/
│   │   ├── client.go               # Internal LLM judge caller (dogfoods own router)
│   │   └── prompts/                # Judge prompt templates per dimension
│   ├── events/
│   │   ├── aiqg_v1.go              # com.tas.aiqg.{request,response}.v1 types
│   │   └── aiqg_publisher.go       # Emits to tas.aiqg.* topics
│   └── config/
│       └── aiqg_config.go         # Config.AIQG nested block (zero value = disabled)
├── pkg/
│   └── clear/
│       ├── scorer.go               # CLEAR composite + per-dimension scorers
│       ├── cost.go                 # ActualCost(usage, pricing) Cost
│       ├── cost_decomposer.go      # direct / induced / genuine waste split
│       ├── latency_score.go        # SCR + p50/p95/p99 → score
│       ├── efficacy_score.go       # structural validity + hedge → score
│       ├── assurance_score.go      # PAS from tags → score
│       ├── reliability_score.go    # pass@k heuristic (partial in MVP)
│       ├── composite.go            # weighted composite
│       └── thresholds.go           # Healthy/Marginal/Failing buckets
├── configs/
│   └── aiqg/
│       ├── starter-deployment.yaml  # K8s manifest for external ingress
│       └── pricing/
│           ├── openai.yaml          # vendor pricing tables (versioned)
│           └── anthropic.yaml
└── docs/
    └── AIQG-EXTENSION.md           # this file
```

Sibling repo `Gatekeeper/`:

```
Gatekeeper/configs/rules/
├── aiqg_workflows.yaml             # workflow classifier (§9)
├── aiqg_context_antipatterns.yaml
├── aiqg_prompt_antipatterns.yaml
├── aiqg_output_antipatterns.yaml
├── aiqg_behavioral_signals.yaml
├── aiqg_clear_assurance.yaml       # NIST AI RMF → CLEAR Assurance mapping
└── aiqg_starter_bundles.yaml       # 4 starter bundles (production_strict, etc.)
```

---

4. Path A Authentication

4.1 Token validation

AIQG tokens are issued by `aiqg-dashboard-be` in the format `tas_qg_live_<base64>` (or `tas_qg_test_*` for non-prod). The proxy validates them by calling the dashboard's `POST /internal/auth/validate` endpoint, cached in Redis.

```go
// internal/aiqg/auth_pathA.go
type Account struct {
    AccountID         uuid.UUID
    TenantID          uuid.UUID
    SourceApp         string
    Region            string
```

```
    ScoringWeights    map[string]float64
    PayloadRetention  string
    Quotas            *Quotas
}

func ValidateAIQGToken(ctx context.Context, token string) (*Account, error) {
    // 1. Redis lookup: "aiqg:token:" + sha256(token)
    // 2. On miss: call aiqg-dashboard-be /internal/auth/validate
    // 3. Cache 60s on hit; 5s on miss (rate-limit failed lookups)
    // 4. Return resolved Account or error
}
```

4.2 Vendor key flow-through

Per build-vs-reuse §7.3: in AIQG mode, the inbound `Authorization: Bearer sk-...` header is forwarded to the vendor **unchanged**. The proxy never persists it.

The mechanism: `internal/aiqg/bearer_context.go` exposes a context value that the existing OpenAI/Anthropic providers read:

```
// internal/aiqg/bearer_context.go
type bearerKey struct{}

func WithBearer(ctx context.Context, token string) context.Context {
    return context.WithValue(ctx, bearerKey{}, token)
}

func BearerFromContext(ctx context.Context) (string, bool) {
    s, ok := ctx.Value(bearerKey{}).(string)
    return s, ok && s != ""
}
```

The existing provider implementations are modified **inside their method bodies only** — signatures unchanged:

```
// internal/providers/openai/provider.go (existing file, body-only change)
func (p *Provider) ChatCompletion(ctx context.Context, req *types.ChatRequest) (*types.ChatResp
    // NEW lines (added inside body):
    if override, ok := aiqg.BearerFromContext(ctx); ok {
        // build a per-request client with the override key
        client := p.clientWithKey(override)
        return p.doChatCompletion(ctx, client, req)
    }
    // EXISTING lines unchanged:
    return p.doChatCompletion(ctx, p.client, req)
}
```

The new helper `p.clientWithKey(override)` is private and additive. The existing `p.client` (built from config) stays as-is and is the path used by every internal caller.

4.3 Strict ingress

The customer-facing ingress at `gateway.aiqg.tas.io` runs the same binary with `Config.AIQG.StrictIngress=true`. In strict mode:

```
// in the top-level handler chain, only when Config.AIQG.StrictIngress is set
if cfg.AIQG.StrictIngress {
    if r.Header.Get("TAS-Auth") == "" || r.Header.Get("Authorization") == "" {
        respondPathAAuthRejected(w, r) // 401 with diagnostic body
        emitAuditEntry(r, "path_a_auth_rejected")
        return
    }
}
```

Internal ingress (the existing in-cluster service) runs with `StrictIngress=false` — current behavior preserved.

The diagnostic 401 body explains which header is missing:

```
{
  "error": "path_a_auth_rejected",
  "message": "AIQG ingress requires both TAS-Auth and Authorization headers. See https://docs.a
  "missing_headers": ["TAS-Auth"]
}
```

---

5. Per-Chunk Timing Capture

5.1 HTTP client instrumentation

`internal/instrumentation/httptrace.go` wraps the outbound HTTP client used by providers:

```
type Timing struct {
    // Front-half checkpoints stamped by handlers along the inbound path.
    // See §5.3 for the stamp-call locations.
    checkpoints                    map[string]time.Time
    mu                             sync.Mutex

    // Outbound HTTP-level checkpoints populated by net/http/httptrace.
    DNSStart, DNSDone              time.Time
    ConnectStart, ConnectDone      time.Time
    TLSStart, TLSDone              time.Time
    GotConn                        time.Time
    WroteRequest                   time.Time
    GotFirstResponseByte           time.Time   // TTFB — HTTP response headers arrival

    // Streaming-level checkpoints populated by chunk_stamp.go.
    Chunks                         []time.Time // every chunk timestamp
    firstContentAt                 time.Time   // TTFT — first chunk with non-empty content de
    ResponseComplete               time.Time
}

func InstrumentedClient(base *http.Client) *http.Client {
    return &http.Client{
        Transport: &tracingTransport{base: base.Transport},
        Timeout:   base.Timeout,
    }
}
```

```go
type tracingTransport struct{ base http.RoundTripper }

func (t *tracingTransport) RoundTrip(req *http.Request) (*http.Response, error) {
    timing := TimingFromContext(req.Context())
    if timing == nil {
        // not AIQG mode → bypass instrumentation entirely
        return t.base.RoundTrip(req)
    }
    trace := &httptrace.ClientTrace{
        DNSStart:             func(httptrace.DNSStartInfo)  { timing.DNSStart = time.Now() },
        DNSDone:              func(httptrace.DNSDoneInfo)   { timing.DNSDone = time.Now() },
        ConnectStart:         func(string, string)          { timing.ConnectStart = time.Now()
        ConnectDone:          func(string, string, error)   { timing.ConnectDone = time.Now() }
        TLSHandshakeStart:    func()                        { timing.TLSStart = time.Now() },
        TLSHandshakeDone:     func(tls.ConnectionState, error) { timing.TLSDone = time.Now() },
        GotConn:              func(httptrace.GotConnInfo)   { timing.GotConn = time.Now() },
        WroteRequest:         func(httptrace.WroteRequestInfo) { timing.WroteRequest = time.Now
        GotFirstResponseByte: func()                        { timing.GotFirstResponseByte = tim
    }
    req = req.WithContext(httptrace.WithClientTrace(req.Context(), trace))
    return t.base.RoundTrip(req)
}
```

5.2 Chunk stamping and content-first detection in the streaming loop

Per build-vs-reuse §2.3, **`types.ChatChunk` is NOT extended**. Chunk timestamps are written to the sidecar collector keyed by request context. The collector also tracks the **first non-empty content chunk** separately from all chunks — this is what produces a correct TTFT (see event-timestamps.md §3 ttft_at).

```go
// internal/instrumentation/chunk_stamp.go

// StampChunk records the receipt time of any SSE chunk (including role/heartbeat openers).
// last_chunk_at is the timestamp of the FINAL StampChunk call before stream close.
func StampChunk(ctx context.Context) {
    t := TimingFromContext(ctx)
    if t == nil {
        return // no-op when not in AIQG mode
    }
    t.Chunks = append(t.Chunks, time.Now())
}

// StampFirstContent records the first time the stream loop encounters a chunk
// whose parsed content delta is non-empty. Idempotent: subsequent calls are no-ops.
// This is TTFT — distinct from TTFB which fires on HTTP response headers.
func StampFirstContent(ctx context.Context) {
    t := TimingFromContext(ctx)
    if t == nil || !t.firstContentAt.IsZero() {
        return // no-op when not in AIQG mode or already stamped
    }
    t.firstContentAt = time.Now()
}
```

Existing stream loops gain two lines — both inside the loop body; signatures and `ChatChunk`

struct unchanged:

```
// internal/providers/openai/provider.go (existing file, body-only change)
for {
    resp, err := stream.Recv()
    if errors.Is(err, io.EOF) { break }
    if err != nil { return err }

    instrumentation.StampChunk(ctx) // every chunk (no-op when not AIQG)

    // TTFT: first chunk whose content delta is non-empty
    if chunkHasNonEmptyContent(resp) {
        instrumentation.StampFirstContent(ctx) // idempotent
    }

    chunks <- convertChunk(resp)
}
```

`chunkHasNonEmptyContent` is provider-specific:

Provider	`chunkHasNonEmptyContent` returns true when
OpenAI	`resp.Choices[0].Delta.Content != ""` OR `len(resp.Choices[0].Delta.ToolCalls) > 0`
Anthropic	the event is `content_block_delta` with `delta.type ∈ {"text_delta","input_json_delta"}` AND non-empty payload

Why this matters: the prior design (`TTFTAt = t.firstChunkOrZero()`) stamped TTFT on the very first `stream.Recv()`, which for OpenAI is typically a role announcement (`delta.role == "assistant"`, empty content) and for Anthropic is `message_start` with no token content. Result: TTFT was 50–200ms too low and CLEAR Latency scores were systematically flattering. This is fixed by content-first detection in the loop. Architect-review finding §4 Risk #5.

5.3 Front-half stamping (auth / headers / scan / policy / emit)

Per the corrected `gateway_overhead_ms` formula (see event-timestamps.md §2.2), five intermediate checkpoints are recorded along the inbound code path. Each landing site is a single-line `instrumentation.StampXxx(ctx)` call in the file that completes the corresponding phase. All are no-ops when context is not AIQG-mode.

Checkpoint	Stamped at	File
`request_received_at`	top of the request handler chain (also the anchor for the timing collector)	`internal/middleware/aiqg_headers.go` (request entry)
`auth_validated_at`	after `ValidateAIQGToken()` returns success	`internal/aiqg/auth_pathA.go`
`headers_parsed_at`	after the loop that parses + `Del()`s all `TAS-*` headers completes	`internal/middleware/aiqg_headers.go` (after strip)

Checkpoint	Stamped at	File
`scan_complete_at`	after the single Hyperscan invocation that runs all inbound rule packs	`internal/workflow/classifier.go`
`policy_resolved_at`	after `policy.Resolve()` returns	`internal/policy/resolver.go`
`request_event_emitted_at`	after `kafkaProducer.Produce()` returns (success or queued; not waiting for `acks=all`)	`internal/events/aiqg_publisher.go`
`request_forwarded_at`	just before the HTTP client writes the first byte to the upstream connection (also captured by `httptrace.WroteRequest` for cross-check)	`internal/providers/{openai,anthropic}/p`

The stamping helpers:

```go
// internal/instrumentation/timing.go
func StampAuthValidated(ctx context.Context)       { stamp(ctx, "auth_validated_at") }
func StampHeadersParsed(ctx context.Context)       { stamp(ctx, "headers_parsed_at") }
func StampScanComplete(ctx context.Context)        { stamp(ctx, "scan_complete_at") }
func StampPolicyResolved(ctx context.Context)      { stamp(ctx, "policy_resolved_at") }
func StampRequestEventEmitted(ctx context.Context) { stamp(ctx, "request_event_emitted_at") }
func StampRequestForwarded(ctx context.Context)    { stamp(ctx, "request_forwarded_at") }

func stamp(ctx context.Context, field string) {
    t := TimingFromContext(ctx)
    if t == nil { return } // no-op when not AIQG-mode
    t.mu.Lock()
    t.checkpoints[field] = time.Now()
    t.mu.Unlock()
}
```

5.4 Timing snapshot for event emission

When the request closes, the timing collector is read to produce an `EventTimestamps` record matching the schema in event-timestamps.md:

```go
func (t *Timing) Snapshot() events.EventTimestamps {
    var p50, p95, p99 float64
    if len(t.Chunks) >= 2 {
        deltas := make([]float64, 0, len(t.Chunks)-1)
        for i := 1; i < len(t.Chunks); i++ {
            deltas = append(deltas, float64(t.Chunks[i].Sub(t.Chunks[i-1]).Milliseconds()))
        }
        sort.Float64s(deltas)
        p50, p95, p99 = pct(deltas, 0.5), pct(deltas, 0.95), pct(deltas, 0.99)
    }
    return events.EventTimestamps{
        RequestReceivedAt:       t.checkpoints["request_received_at"],
        AuthValidatedAt:         t.checkpoints["auth_validated_at"],
```

```
        HeadersParsedAt:         t.checkpoints["headers_parsed_at"],
        ScanCompleteAt:          t.checkpoints["scan_complete_at"],
        PolicyResolvedAt:        t.checkpoints["policy_resolved_at"],
        RequestEventEmittedAt:   t.checkpoints["request_event_emitted_at"],
        RequestForwardedAt:      t.checkpoints["request_forwarded_at"],
        DNSResolvedAt:           t.DNSDone,
        TCPConnectedAt:          t.ConnectDone,
        TLSHandshakeCompleteAt:  t.TLSDone,
        TTFBAt:                  t.GotFirstResponseByte,        // first HTTP response byte
        TTFTAt:                  t.firstContentAt,              // first non-empty content delt
        LastChunkAt:             t.lastChunkOrZero(),
        ResponseCompleteAt:      t.ResponseComplete,
        ChunkCount:              len(t.Chunks),
        InterTokenLatencyP50Ms:  p50,
        InterTokenLatencyP95Ms:  p95,
        InterTokenLatencyP99Ms:  p99,
    }
}
```

`TTFTAt` is sourced from `t.firstContentAt` (set by `StampFirstContent`), NOT from `t.firstChunkOrZero()`. The old code path that conflated TTFT with the first SSE chunk is deleted.

5.5 SLO breach detection and emission

At the end of every AIQG-mode request — success, error, or client disconnect — the handler calls `timing.CheckSLO(ctx, cfg.AIQG.GatewayOverheadSLO)`. This:

1. Computes `gateway_overhead_ms` from the snapshot using the same formula the TimescaleDB generated column uses
2. If overhead ≤ target, return silently (the snapshot's `slo_breached` field stays false in the response event)
3. If overhead > target, identifies the **dominant phase** by taking the largest of {auth, headers, scan, policy, emit, forward_prep, egress} sub-costs
4. Increments `aiqg_slo_breach_total{slo, dominant_phase, vendor, account_id, cache_state}`
5. Emits a structured WARN log via the standard zap logger (sampled per `cfg.AIQG.SLOBreachSampleRate`, default 1.0)
6. Publishes an audit-log entry of type `slo_breach_observed` to Kafka with the full phase breakdown

The full payload schema and standing alert rule are defined in event-timestamps.md §4.6. The runbook for breaches lives at `runbooks/gateway-overhead-breach.md` in this design package (TODO: author).

---

6. TAS-* Header Taxonomy

`internal/middleware/aiqg_headers.go` parses and strips all `TAS-*` headers before the request is forwarded upstream. Vendors never see them.

Header	Action
`TAS-Auth`	Validated by `auth_pathA.go`; account stashed in ctx

Header	Action
`TAS-Policy`	Parsed as comma-separated rule names; passed to policy resolver
`TAS-Policy-Bundle`	Single bundle name; passed to policy resolver
`TAS-Workflow`	Customer-provided workflow override; stashed in ctx for classifier to honor
`TAS-Upstream-Authorization`	Per-request vendor key override; `aiqg.WithBearer(ctx, value)` if present
`TAS-Trace`	When `1`, the response includes `TAS-Trace-Result` header (base64 event JSON)
`TAS-Dry-Run`	Marks the request as dry-run; policies evaluate but don't enforce

All `TAS-*` headers are stripped via `req.Header.Del()` after parsing, before the request hits `httputil.ReverseProxy` (or its equivalent in the providers).

---

7. Workflow Classification, Sampling, Policy Resolution

7.1 Workflow classifier

`internal/workflow/classifier.go` runs the Gatekeeper scanner with the `aiqg_workflows.yaml` rule pack against the inbound request. Per workflow-classification.md, output is one of `single_turn_qa`, `rag`, `agentic`, `summarization`, `code_generation`, `classification_extraction`, or `unknown`.

```
func Classify(ctx context.Context, req *types.ChatRequest) workflow.Result {
    if customer := ctx.Value(workflowOverrideKey{}); customer != nil {
        return workflow.Result{
            Type:       customer.(string),
            Confidence: 1.0,
            Source:     "customer_override_header",
        }
    }
    scanResult := gatekeeperScanner.Scan(serializeForClassifier(req), "aiqg_workflows")
    return interpretScanResult(scanResult)
}
```

The classification result is written to `inferred_labels.workflow_type` on the AIQG event and to the `workflow:*` tag in tag-set.md.

7.2 Sampling decision

`internal/sampling/stratified.go` decides whether this request becomes an LLM-judge sample, stratified by workflow + customer + recent anomaly history. Deterministic sampling (token counts, schema validation, Hyperscan tagging) always runs at 100%.

```
func ShouldSample(ctx context.Context, classification workflow.Result, accountID uuid.UUID) sam
    rate := lookupSampleRate(classification.Type, accountID) // 5-10% small, 1% large
    if rateLimitedByAnomaly(ctx) {
        rate *= 3 // sharpen sampling when degradation detected
```

```
    }
    if rand.Float64() < rate {
        return sampling.Decision{LLMJudge: true, JudgeModel: "small"}
    }
    return sampling.Decision{}
}
```

7.3 Policy resolution

`internal/policy/resolver.go` runs the algorithm specified in route-rule.md:

```
1. If request has TAS-Policy or TAS-Policy-Bundle header → use that bundle
2. Else: enumerate enabled route rules for tenant ordered by priority, take first match
3. Else: account-default bundle
4. Else: pure pass-through (measurement only)
```

Resolved bundle ID is stamped into the AIQG request event with `policy_resolution_source` set to one of `header_override`, `route_rule`, `account_default`, or `pass_through`.

Resolution is cached in Redis (`aiqg:resolve:{tenant_id}:{fingerprint}`, 60s TTL); cache invalidates on Kafka events `com.tas.aiqg.bundle.updated.v1` and `com.tas.aiqg.route_rule.changed.v1`.

---

8. CLEAR Scoring (Gateway-Side)

Per build-vs-reuse §7.2, scoring happens **in the gateway at request close**, in Go. Each dimension has a dedicated scorer:

```
// pkg/clear/scorer.go
type Inputs struct {
    Timing        events.EventTimestamps
    Usage         events.TokenAccounting
    Validity      bool
    Tags          []string
    WorkflowType  string
    AccountWeights map[string]float64
    AccountThresholds map[string]Threshold
}

type Scores struct {
    Cost, Latency, Efficacy, Assurance, Reliability uint8
    Composite                                       uint8
    Version                                         string
    WeightsUsed                                     map[string]float64
    ThresholdsUsed                                  map[string]Threshold
}

func Score(in Inputs) Scores {
    s := Scores{Version: ScoringVersion}
    s.Cost        = ScoreCost(in.Usage, in.WorkflowType, in.AccountThresholds["cost"])
    s.Latency     = ScoreLatency(in.Timing, in.WorkflowType, in.AccountThresholds["latency"])
    s.Efficacy    = ScoreEfficacy(in.Validity, in.Tags, in.AccountThresholds["efficacy"])
    s.Assurance   = ScoreAssurance(in.Tags, in.AccountThresholds["assurance"])
    s.Reliability = ScoreReliabilityHeuristic(in.Tags, in.AccountThresholds["reliability"])
```

```
    s.Composite   = ComposeWeighted(s, in.AccountWeights)
    s.WeightsUsed = in.AccountWeights
    s.ThresholdsUsed = in.AccountThresholds
    return s
}
```

`ScoringVersion = "clear-v1.0"` for MVP. Per build-vs-reuse §7.2 mitigation, this string is embedded in every emitted event, enabling future re-scoring by a Spark job if formulas evolve.

Cost scorer detail

`pkg/clear/cost.go` provides `ActualCost(usage, pricing) Cost` — a **new function** that returns post-response actual cost (per build-vs-reuse §1.2). The existing `EstimateCost()` is untouched.

`pkg/clear/cost_decomposer.go` produces the three-category split (direct payload waste / induced output waste / genuine post-model waste) per token-accounting.md. The decomposition uses heuristics in MVP:

- **Direct payload waste**: sample-based embedding similarity between context blocks and output. Tokens in blocks with similarity < threshold attributed to waste.
- **Induced output waste**: estimated from `inferred_labels.is_retry_of_previous` × cost of failed prior request.
- **Genuine post-model waste**: residual after the two above; bounded by `actual_cost - estimated_addressable`.

Decomposition results are conservative: missing components default to null, not zero.

---

9. AIQG CloudEvent Emission

9.1 Event types

Two new CloudEvent types are defined in `internal/events/aiqg_v1.go`:

```
const (
    EventTypeAIQGRequest  = "com.tas.aiqg.request.v1"
    EventTypeAIQGResponse = "com.tas.aiqg.response.v1"
)

type AIQGRequestEvent struct {
    RequestEventID    uuid.UUID       `json:"request_event_id"`
    TenantID          uuid.UUID       `json:"tenant_id"`
    AIQGAccountID     uuid.UUID       `json:"aiqg_account_id"`
    ReceivedAt        time.Time       `json:"received_at"`
    Vendor            string          `json:"vendor"`
    Endpoint          string          `json:"endpoint"`
    Model             string          `json:"model"`
    SourceApp         string          `json:"source_app"`
    TasAuthTokenID    uuid.UUID       `json:"tas_auth_token_id"`
    Streaming         bool            `json:"streaming"`
    RequestStructure  RequestStructure  `json:"request_structure"`
    InferredLabels    InferredLabels    `json:"inferred_labels"`
    AppliedBundleID   *uuid.UUID      `json:"applied_bundle_id"`
    PolicyResolution  string          `json:"policy_resolution_source"`
```

```
    DryRun            bool            `json:"dry_run"`
    Tags              []string        `json:"tags"`
}

type AIQGResponseEvent struct {
    ResponseEventID    uuid.UUID          `json:"response_event_id"`
    RequestEventID     uuid.UUID          `json:"request_event_id"`
    TenantID           uuid.UUID          `json:"tenant_id"`
    AIQGAccountID      uuid.UUID          `json:"aiqg_account_id"`
    CompleteAt         time.Time          `json:"complete_at"`
    Status             string             `json:"status"`
    HTTPStatus         int                `json:"http_status"`
    FinishReason       string             `json:"finish_reason"`
    Streamed           bool               `json:"streamed"`
    ChunkCount         int                `json:"chunk_count"`
    ResponseStructure  ResponseStructure  `json:"response_structure"`
    TokenAccounting    TokenAccounting    `json:"token_accounting"`
    Timestamps         EventTimestamps    `json:"timestamps"`
    ClearScores        clear.Scores       `json:"clear"`
    Tags               []string           `json:"tags"`
    SampledForJudge    bool               `json:"sampled_for_llm_judge"`
    PayloadRetained    bool               `json:"payload_retained"`
    PayloadStorageURI  string             `json:"payload_storage_uri,omitempty"`
}
```

Field shapes match the aiqg/ data-model docs.

9.2 Kafka topics

Topic	Purpose	Partition key
`tas.aiqg.request.v1`	Per-request capture (AIQG mode only)	`tenant_id`
`tas.aiqg.response.v1`	Per-response capture	`tenant_id`
`tas.aiqg.findings.v1`	Gatekeeper findings stream (existing pattern, namespaced)	`tenant_id`
`tas.aiqg.bundle.updated.v1`	Cache invalidation	none
`tas.aiqg.route_rule.changed.v1`	Cache invalidation	none
`tas.aiqg.report.generate.v1`	Report generation job queue (consumed by dashboard-be)	`tenant_id`

The existing `com.tas.activity.llm.{request,response}` events on topic `tas.activity.llm` continue to be emitted unchanged for internal-routing traffic. Per build-vs-reuse §1.2 and wire-compat checklist §10.4.

9.3 Emission flow

```
request received
  ├── AIQG mode? → no  → existing path (emit com.tas.activity.llm.* unchanged)
  ├── AIQG mode? → yes →
  │     ├── attach instrumentation.Timing to ctx
  │     ├── validate TAS-Auth, attach Account to ctx
```

```
│      ├── parse + strip TAS-* headers
│      ├── workflow.Classify(ctx, req)
│      ├── sampling.ShouldSample(ctx, classification, accountID)
│      ├── policy.Resolve(ctx, req)
│      ├── apply policy rules (tag, block, redact as configured)
│      ├── emit AIQGRequestEvent → tas.aiqg.request.v1
│      ├── forward to vendor (with bearer override from ctx)
│      ├── stream response → instrumentation.StampChunk on each chunk
│      ├── on completion: read timing, compute usage, score CLEAR
│      └── emit AIQGResponseEvent → tas.aiqg.response.v1
```

---

10. Config Schema

`Config.AIQG` is a new nested struct. **Empty/zero value disables every AIQG feature** — existing deployments using current config files boot identically to today.

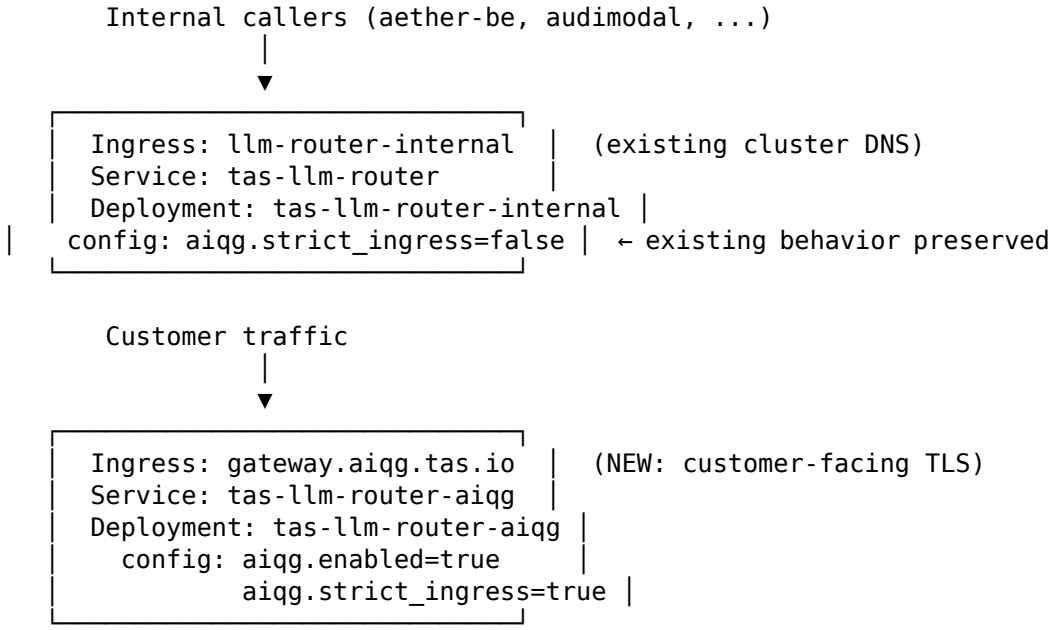
```go
// internal/config/aiqg_config.go
type AIQGConfig struct {
    Enabled              bool          `yaml:"enabled" env:"AIQG_ENABLED" default:"false"`
    StrictIngress        bool          `yaml:"strict_ingress" env:"AIQG_STRICT_INGRESS" default
    DashboardURL         string        `yaml:"dashboard_url" env:"AIQG_DASHBOARD_URL"`
    DashboardAuthToken   string        `yaml:"-" env:"AIQG_DASHBOARD_AUTH_TOKEN"`
    KafkaTopicPrefix     string        `yaml:"kafka_topic_prefix" default:"tas.aiqg"`
    Neo4jURI             string        `yaml:"neo4j_uri" env:"AIQG_NEO4J_URI"`
    RedisAddr            string        `yaml:"redis_addr" env:"AIQG_REDIS_ADDR"`
    TimingBufferSize     int           `yaml:"timing_buffer_size" default:"256"`
    JudgeModel           string        `yaml:"judge_model" default:"gpt-4o-mini"`
    PricingDir           string        `yaml:"pricing_dir" default:"configs/aiqg/pricing"`

    // GatewayOverheadSLO is the per-request gateway-overhead target.
    // Default 50ms is intentionally aspirational — see event-timestamps.md §4.3.
    // Every request whose computed overhead exceeds this is recorded as an SLO
    // breach (Prometheus counter + structured log + audit-log entry); the
    // standing alert AIQGGatewayOverheadBreachRateHigh fires when per-tenant
    // breach rate exceeds 1% over 5min.
    GatewayOverheadSLO   time.Duration `yaml:"gateway_overhead_slo" default:"50ms"`

    // SLOBreachSampleRate controls how often a breach emits a log line and an
    // audit-log entry. 1.0 = every breach (the Prometheus counter is always
    // emitted at full rate regardless of this setting; cardinality is
    // bounded by labels). Lower in sustained-breach hotspots to bound Loki
    // ingest and audit-table growth.
    SLOBreachSampleRate  float64       `yaml:"slo_breach_sample_rate" default:"1.0"`
}

// in the existing Config struct (additive field):
type Config struct {
    // ... existing fields unchanged ...
    AIQG AIQGConfig `yaml:"aiqg"`
}
```

Existing `config.yaml` files (no `aiqg:` block) parse cleanly with all AIQG features off.

---

11. Deployment Topology

The same binary supports both internal and external use via two Deployments behind two Ingresses:



Both Deployments build from the same Docker image. Configuration is environment-specific. This gives operational separation (independent scaling, separate K8s NetworkPolicies, separate resource budgets) without forking the code.

Per CLAUDE.md K8s policy, the new `tas-llm-router-aiqg` Deployment ships as BestEffort QoS initially; profile under representative load before adding requests/limits.

---

12. Wire-Compat Test Surface

Per build-vs-reuse §10, these tests gate every PR touching this extension. New files in this repo:

```
tas-llm-router/
├── internal/
│   └── contract/
│       ├── http_contract_test.go      # Existing endpoint contracts (§10.1)
│       ├── struct_snapshot_test.go    # Struct JSON shape (§10.3)
│       ├── cloudevent_schema_test.go  # com.tas.activity.llm.* schema (§10.4)
│       ├── metric_set_test.go         # llm_router_* metrics (§10.8)
│       ├── default_config_boot_test.go # zero-AIQG-config boot (§10.9)
│       └── interface_stability_test.go # LLMProvider + Publisher (§10.2, §10.11)
├── testdata/
│   ├── contract/                       # canned request/response fixtures
│   └── events/                         # event schema JSON
└── Makefile                            # add `make test-contract` target
```

Detail of each test is in the build-vs-reuse §10 sub-sections — this file does not duplicate them.

---

13. Phasing

Phase	Capability	Reference
MVP (Phase 1)	All of §2–§11 except payload reduction, route policy editor UI, Bedrock/Vertex	spec §5.2
Phase 2	Payload reduction, route policy editor UI, Path B (stored keys), Bedrock/Vertex, drift alerting, full Efficacy + Reliability via conversation threading	spec §5.3
Phase 3	Thin client SDK, outcome webhook, custom rule packs, eval set management, compliance-vertical bundles, RBAC	spec §5.4

The MVP scope of this extension lands the data flowing into Kafka and the AIQG CloudEvent schema stable. `aiqg-dashboard-be` and `aiqg-ui` can ship in parallel and consume the events as soon as they appear.

---

14. Open Questions

- **Reliability heuristic for MVP.** Spec §2.5 defers full pass@k to Phase 2. The MVP heuristic uses structural consistency + retry-rate proxies. Final formula needs sign-off from the methodology owner. Default: low confidence; report shows "partial" rather than a number until Phase 2.
- **Judge model choice.** Defaults to `gpt-4o-mini`. Anthropic equivalents may be cheaper at comparable judge quality; defer the choice to a Phase 1 cost study.
- **Pricing table format & update cadence.** YAML in `configs/aiqg/pricing/`, versioned by date. Updates ship as new versions; old prices preserved. Update cadence: weekly via a job that scrapes vendor pricing pages — TBD whether this is in scope for MVP or operator-managed.

---

15. Related Documents

- AIQG data models — the 16 model docs underpinning every emission
- build-vs-reuse.md — the master plan; this doc is the gateway-side instantiation
- source-spec-v0.2.md — the product spec
- tas-llm-router/docs/api-reference.md — existing API surface (unchanged by this work)
- Gatekeeper/CLAUDE.md — the scanner library this work depends on